

# Projektowanie Systemów Informatycznych

**Inżynieria oprogramowania** dotyczy stosowania empirycznego, naukowego podejścia do znajdowania wydajnych i ekonomicznych rozwiązań praktycznych problemów w dziedzinie oprogramowania.

**Projektowanie systemów informatycznych** to proces definiowania architektury, komponentów, modułów, interfejsów i danych tak, aby system spełniał określone wymagania użytkownika końcowego. Celem jest stworzenie dobrze zorganizowanej struktury, która spełnia zamierzony cel, jednocześnie biorąc pod uwagę takie czynniki, jak skalowalność, łatwość utrzymania i wydajność.

## Proces tworzenia oprogramowania

Każde oprogramowanie wytwarza się w czterech etapach:

**Planowanie → Analiza wymagań → Projektowanie → Implementacja**

### Etap 1. Planowanie (identyfikacja celów)

Odpowiedź na pytania: **po co** tworzymy to oprogramowanie i **jak** ono powstanie.  
Ustala się wartości biznesowe projektu, wykonalność wdrożenia oraz dostępne zasoby.

### Etap 2. Analiza wymagań (określenie funkcji systemu)

Ustala się: **kto** będzie używał systemu, **co** system ma robić, **gdzie i kiedy** będzie używany.  
System opisywany jest jako czarna skrzynka (black box), bez narzucania konkretnych technologii.

### Etap 3. Projektowanie (architektura i struktura danych)

Pierwsza decyzja o tym, jak system ma działać i jak ma spełniać wymagania użytkownika.  
Tu powstaje koncepcja wewnętrznej logiki systemu, jego architektura, interfejsy (w tym użytkownika).

Obecnie stosuje się metody obiektowe<sup>1</sup>, a projekt często stanowi abstrakcję przyszłego systemu: jest to najczęściej forma opisu słownego oraz wizualizacji (rysunki, schematy, diagramy).

### Etap 4. Implementacja (kodowanie)

Końcowa faza procesu tworzenia oprogramowania.  
Podczas implementacji zostaje zbudowana baza danych w wybranej technologii implementacyjnej i wytworzony kod aplikacji.

(tutaj się kończy samo wytwarzanie oprogramowania, ale zgodnie z fazami Cyklu życia systemu informatycznego, po implementacji dochodzi: Testowanie, Wdrożenie, Eksploatacja, Wycofanie. Czyli po wykonaniu testów systemu można wdrożyć system w warunkach rzeczywistych i sukcesywnie utrzymywać go w ciągłej pracy, aż do jego finalnego wycofania).

<sup>1</sup> Programowanie strukturalne a obiektowe. Programowanie strukturalne skupia się na pisaniu funkcji. W programowaniu obiektowym najważniejsze są dane.

# Cykl życia systemu informatycznego

---

## Etap 1. Planowanie (identyfikacja celów)

Odpowiedź na pytania: **po co** tworzymy to oprogramowanie i **jak** ono powstanie.  
Ustala się wartości biznesowe projektu, wykonalność wdrożenia oraz dostępne zasoby.

**Przykład:** Firma planuje stworzyć system zarządzania klientami CRM, opisuje jego najważniejsze cechy i zespół zaangażowany w jego wdrożenie.

## Etap 2. Analiza wymagań (określenie funkcji systemu)

Ustala się: **kto** będzie używał systemu, **co** system ma robić, **gdzie i kiedy** będzie używany.  
System opisywany jest jako czarna skrzynka (black box), bez narzucania konkretnych technologii.

**Przykład:** Firma sprawdza, czy zbudowanie systemu CRM jest warte kosztów i wysiłku oraz czy może on przynieść oczekiwane korzyści i czy spełnia oczekiwania użytkowników rynku.

## Etap 3. Projektowanie (architektura i struktura danych)

Pierwsza decyzja o tym, jak system ma działać i jak ma spełniać wymagania użytkownika.  
Tu powstaje koncepcja wewnętrznej logiki systemu, jego architektura, interfejsy (w tym użytkownika).

**Przykład:** Powstaje szczegółowy plan dla deweloperów, jak ma działać i wyglądać system CRM - analogicznie jak plany architektoniczne przed budową domu.

## Etap 4. Implementacja (kodowanie)

Końcowa faza procesu tworzenia oprogramowania.  
Podczas implementacji zostaje zbudowana baza danych w wybranej technologii implementacyjnej i wytworzony kod aplikacji.

**Przykład:** Kodowane są funkcje systemu CRM: profile klientów, pulpity nawigacyjne itp.

## Etap 5. Testowanie

Weryfikacja, czy system spełnia określone wymagania i czy wszystko działa zgodnie z projektem.

**Przykład:** System CRM poddawany jest różnym testom, np. testy jednostkowe, testy integracyjne i testy akceptacji użytkownika, aby zapewnić jego funkcjonalność, wydajność i bezpieczeństwo.

## Etap 6. Wdrożenie (uruchomienie systemu na produkcji, deployment/release)

Przeniesienie systemu ze środowiska testowego do produkcyjnego i udostępnienie faktycznym użytkownikom.

## Etap 7. Eksploatacja i utrzymanie (konserwacja, monitorowanie, aktualizacje)

Regularne aktualizacje, poprawki błędów i wsparcie użytkowników w dostosowywaniu systemu do zmieniających się wymagań biznesowych oraz w rozwiązywaniu pojawiających się problemów.

## Etap 8. Wycofanie

Zamknięcie systemu.

## Cykl życia systemu vs. Proces tworzenia systemu

| Aspekt           | Cykl życia systemu informatycznego                                                                 | Proces tworzenia oprogramowania                                       |
|------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Definicja</b> | Kompleksowe ramy obejmujące cały proces rozwoju systemu.                                           | Podzbiór zajmujący się konkretnie projektowaniem i wykonaniem systemu |
| <b>Zakres</b>    | Cały cykl życia: od planowania, przez uruchomienie, do wycofania.                                  | Przed wszystkim aspekty projektowe systemu.                           |
| <b>Fazy</b>      | Planowanie, analiza, projektowanie, implementacja, testowanie, wdrożenie, eksploatacja, wycofanie. | Planowanie, analiza wymagań, projektowanie, implementacja.            |
| <b>Centrum</b>   | Całościowy proces rozwoju: planowanie, wdrażanie, testowanie, konserwacja.                         | Etap projektowania: szczegółowy opis budowy i działania systemu.      |
| <b>Zamiar</b>    | Prowadzi zespół przez cały proces: od koncepcji po wsparcie powdrożeniowe.                         | Plan budowy systemu w oparciu o określone wymagania projektowe.       |

### Typowe wyzwania projektowe

- Niejasne lub niejednoznaczne wymagania na etapie startu projektu.
- Zmieniające się wymagania w trakcie procesu projektowania.
- Szybki postęp technologiczny utrudniający dobór właściwych narzędzi.
- Złożona integracja komponentów z różnych technologii i platform.
- Ograniczenia budżetowe utrudniające pełną realizację wymagań.

## Cykl życia procesu analizy danych w Data Science

Projektowanie nowoczesnych systemów informatycznych coraz częściej obejmuje wsparcie analizy danych i procesów decyzyjnych opartych na algorytmach data science.

- 1. Zdefiniuj cel** - *Jaki problem staram się rozwiązać?*
- 2. Zgromadź dane i zarządzaj nimi** - *Jakie informacje są mi potrzebne?*
- 3. Zbuduj model** - *Znajdź w danych wzorce prowadzące do rozwiązań.*
- 4. Oceń model i poddaj go krytyce** - *Czy model rozwiązuje mój problem?*
- 5. Zaprezentuj wyniki i udokumentuj je** - *Udowodnij, że możesz rozwiązać problem i pokaż, jak tego dokonasz.*
- 6. Wdróż model** - *Wdróż model w środowisku produkcyjnym i utrzymuj go.*

# Gromadzenie i analiza wymagań funkcjonalnych i нефункциональных w projektowaniu systemów informatycznych

---

Czy pamiętasz Etap 2 z Cyklu życia systemu informatycznego?

## Etap 2. Analiza wymagań (określenie funkcji systemu)

Ustala się: **kto** będzie używał systemu, **co** system ma robić, **gdzie i kiedy** będzie używany. System opisywany jest jako czarna skrzynka (black box), bez narzucania konkretnych technologii.

**Przykład:** Firma sprawdza, czy zbudowanie systemu CRM jest warte kosztów i wysiłku oraz czy może on przynieść oczekiwane korzyści i czy spełnia oczekiwania użytkowników rynku.

**Analiza wymagań** polega na określeniu funkcji systemu.

Spisanie tych wymagań pozwala na stworzenie systemu oprogramowania, który spełnia oczekiwania klienta (zamawiającego) w określonych ramach czasowych i budżetowych oraz jest zgodny z celami systemu zidentyfikowanymi w poprzednim etapie tj. **Etap 1. Planowanie (identyfikacja celów)**.

### WYMAGANIA FUNKCJONALNE

opisują **CO system ma robić**, czyli jakie funkcje ma realizować.

Odpowiadają na pytania: „Co system ma umożliwić?”, „Jakie zadania ma wykonywać?”.

Przykłady wymagań funkcjonalnych:

- Możliwość logowania się użytkowników.
- Wyszukiwanie produktów w sklepie internetowym.
- Generowanie raportów finansowych.

### WYMAGANIA NIEFUNKCJONALNE

opisują **JAK system ma działać**, czyli jakie ma mieć cechy jakościowe.

Odpowiadają na pytania: „Jak system ma działać?”, „Jakie ma mieć właściwości?”.

Przykłady wymagań нефункциональных:

- Wydajność (np. czas odpowiedzi systemu).
- Bezpieczeństwo (np. ochrona danych przed nieautoryzowanym dostępem).
- Niezawodność (np. dostępność systemu przez 24/7).
- Użyteczność (np. łatwość obsługi interfejsu użytkownika).

## Proces gromadzenia i analizy wymagań:

1. **Identyfikacja interesariuszy:** należy zidentyfikować wszystkie osoby/grupy osób, które są zainteresowane systemem lub będą z niego korzystać.
2. **Gromadzenie wymagań:** stosuje się różne techniki odkrywania wymagań: burze mózgów, konsultacje i wywiady z kluczowymi użytkownikami, analiza dokumentów, prototypowanie.
3. **Analiza wymagań:** należy sprawdzić, czy wymagania są kompletne (niczego nie pominęliśmy), jednoznaczne (bez różnych interpretacji), spójne (niesprzeczne), testowalne (mieralne).
4. **Dokumentowanie wymagań:** wymagania są zapisywane w formie dokumentu specyfikacji wymagań, który jest podstawą dla następnego etapu tj. **Etap 3. Projektowanie (architektura i struktura danych)**
5. **Walidacja wymagań:** należy upewnić się, że zgromadzone wymagania są zgodne z rzeczywistymi oczekiwaniami interesariuszy. Wymagania powinny być regularnie weryfikowane i aktualizowane w trakcie trwania projektu.

# Dokumentacja i specyfikacja wymagań w procesie projektowania systemów informatycznych

---

## Dokumentacja wymagań:

- Jest **zbiorem dokumentów**, które opisują wymagania systemu.
- Służy jako punkt odniesienia dla wszystkich etapów projektu.
- Powinna być zrozumiała, spójna i aktualna.
- Obejmuje m.in.:
  - Specyfikację wymagań oprogramowania: **Software Requirements Specification (SRS)**
  - Przypadki użycia (use cases)
  - Scenariusze użytkownika (user stories)

## Specyfikacja wymagań

- Jest formalnym dokumentem, który szczegółowo opisuje wymagania systemu.
- Stanowi podstawę do następnego etapu po Analizie wymagań (tj. Projektowanie).
- Zazwyczaj zawiera:
  - Opis celów systemu.
  - Opis wymagań funkcjonalnych i нефункциональных.
  - Opis interfejsów użytkownika.
  - Opis wymagań dotyczących danych.

## Znaczenie dokumentacji i specyfikacji wymagań:

- Zapewnienie jasności i zrozumienia celów projektu
- Minimalizacja ryzyka nieporozumień i błędów
- Ułatwienie komunikacji między interesariuszami
- Zapewnienie spójności i jakości systemu
- Ułatwienie testowania i utrzymania systemu
- Lepsze zarządzanie projektem, ocena jego kosztów, przewidywanie terminów realizacji oraz ocena zwrotu z inwestycji (ROI)

## Podejście Reproducible Research i jego wpływ na replikację wyników

---

Podejście Reproducible Research to fundament nowoczesnej nauki i inżynierii danych. Wspiera transparentność, pozwala na replikację wyników i ułatwia współpracę naukową i biznesową

Reproducible Research polega na dokumentowaniu kodu, danych i metod  
w sposób umożliwiającym ich powtórzenie.

**Reproducible Research** odnosi się do zestawu praktyk, które mają na celu zapewnienie, że wyniki badań (zwłaszcza empirycznych, opartych na danych) mogą być **powtórzone i zweryfikowane przez inne osoby**.

Najważniejszą ideą RR jest to, że każdy wynik powinien być:

- **Sprawdzalny** - osoba trzecia powinna móc odtworzyć wyniki na podstawie dostarczonych danych i kodu.
- **Transparentny** - kod, dane i metodyka analizy muszą być jawne i zrozumiałe.
- **Udokumentowany** - wyniki analizy powinny być zintegrowane z opisem metod i kodem źródłowym oraz dokumentacją i specyfikacją wymagań.

### Reproducible Research a replikacja wyników

W kontekście nowoczesnych projektów systemów informatycznych, RR umożliwia:

- **Replikację wyników** - ponowne uruchomienie kodu na tych samych danych powinno dać identyczne wyniki.
- **Walidację modeli** - pozwala niezależnie ocenić skuteczność modeli ML/Text Mining.
- **Audyt** - interesariusze systemu mogą przeanalizować pełen przebieg analizy.

## Najważniejsze elementy RR w data science i text mining

| Element          | Znaczenie                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------|
| Kod źródłowy     | Wszystkie skrypty analityczne muszą być czytelne, modularne i udostępnione.                            |
| Dane wejściowe   | Oryginalne dane (lub link do źródła) muszą być dostępne.                                               |
| Środowisko pracy | Narzędzia, biblioteki i wersje pakietów powinny być zdefiniowane (np. <code>sessionInfo()</code> w R). |
| Dokumentacja     | Opis założeń, kroków przetwarzania, metryk i sposobu interpretacji wyników.                            |
| Raport końcowy   | Generowany automatycznie (np. R Markdown, Jupyter Notebook, Quarto).                                   |

## Narzędzia wspierające podejście RR

- **R Markdown / Jupyter Notebook / Quarto** - integracja tekstu opisu i kodu analitycznego do reprodukcji analiz.
- **Git + GitHub** - wersjonowanie kodu i dokumentacji, współpraca zespołowa.
- **Docker / renv / conda** - replikowalne środowiska pracy.
- **Zenodo / OSF.io** - archiwizacja danych i publikacja kodu.

Podejście RR ma wymierny wpływ na nowoczesne systemy informatyczne:

- Zwiększa wiarygodność wyników w nauce i w biznesie.
- Przyspiesza rozwój i debugging dzięki jasnej dokumentacji i modularności kodu.
- Ułatwia współpracę interdyscyplinarną wszystkim członkom zespołu.
- Przygotowuje systemy do audytu i zgodności regulacyjnej, np. w finansach, medycynie.

## Przykład: projekt text mining w R z wykorzystaniem RR

Plik zawiera modularny kod:

- wczytanie danych tekstowych
- preprocessing tekstu (normalizacja, czyszczenie, stemming, tokenizacja)
- zliczanie częstości słów
- wizualizacja
- eksport wyników
- Wszystko udokumentowane w jednym reprodukowalnym pliku HTML / RMarkdown / Jupyter Notebook.
- Udostępnione na profilu GitHub wraz z danymi.



## Metodyki Agile i Waterfall

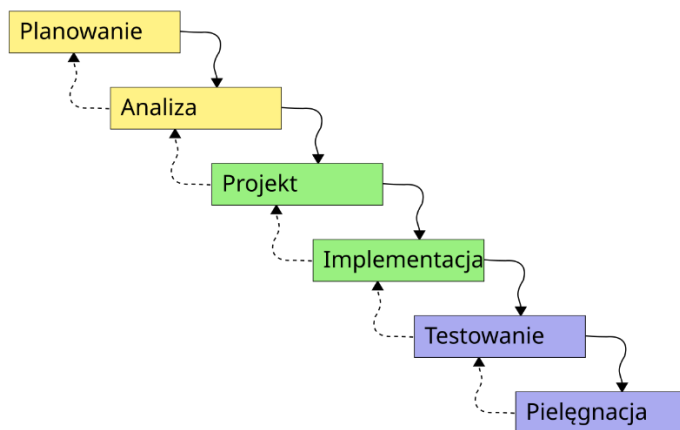
W większości współczesnych procesów wytwarzania oprogramowania, stosuje się jeden z dwóch modeli wytwórczych:

- Model kaskadowy (*waterfall*, wodospadowy)
- Model przyrostowy (*incremental*, iteracyjny); jego rozwinięciem jest model spiralny

### Model kaskadowy

**liniowy i sekwencyjny** - nie można przejść do następnej fazy przed zakończeniem poprzedniej; błąd popełniony w początkowej fazie ma wpływ na całość; łatwy w nadzorowaniu.

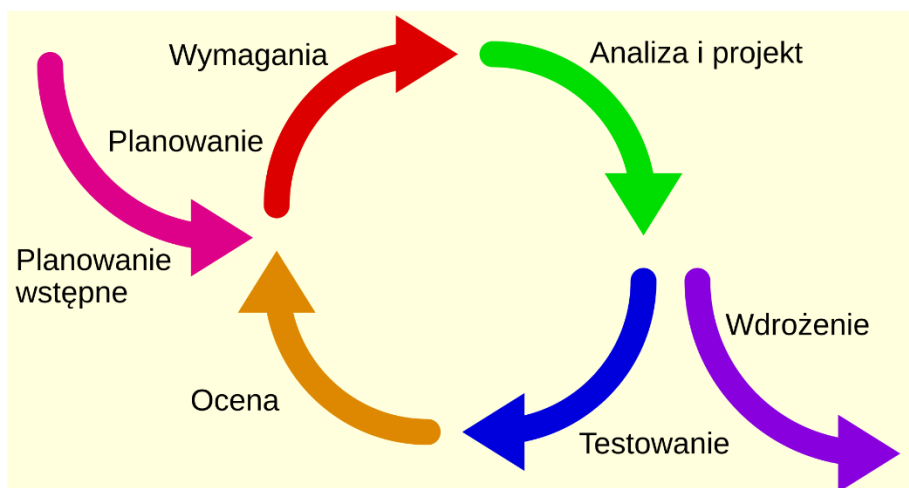
[https://pl.wikipedia.org/wiki/Model\\_kaskadowy](https://pl.wikipedia.org/wiki/Model_kaskadowy)



### Model przyrostowy

**ewolucyjny** - pozwala na powroty do faz poprzedzających, co umożliwia adaptowanie systemu do zmian w wymaganiach; umożliwia korygowanie popełnionych błędów; trudny w nadzorowaniu.

[https://pl.wikipedia.org/wiki/Model\\_przyrostowy](https://pl.wikipedia.org/wiki/Model_przyrostowy)



W praktyce codziennej pracy zespoły projektowe/deweloperskie potrzebują konkretnych wytycznych **"jak należy wytworzyć oprogramowanie"**. Zespół musi wiedzieć, co należy wykonać, w jaki sposób oraz kto powinien za co odpowiadać.

**Modele wytwórcze mają swoje warianty**  
**charakterystyczne dla**  
**odpowiednich metodyk wytwarzania oprogramowania.**

Metodyka to ustandaryzowane podejście do rozwiązywania problemów dla wybranego obszaru.

### **Metodyki kaskadowe Waterfall - tradycyjne**

Wytwarzanie oprogramowania w tradycyjnym podejściu opiera się w uproszczeniu na trzech krokach: zleceniobiorca ustala z klientem co ma zostać zrobione, zespół deweloperski wykonuje projekt, system w całości zostaje przekazany do klienta.

Tutaj każda kolejna faza następuje po kolei jedna za drugą i nie pomija się żadnego etapu. Ewentualne zmiany „w fazie poprzedniej” są trudne do wprowadzenia po rozpoczęciu kolejnej fazy.

### **Metodyki zwinne Agile - nowoczesne**

Sednem metodyk zwinnych Agile jest cykliczność postępu prac nad projektem i oddawanie go klientowi w mniejszych „porcjach”, przyrostowo. Klient widzi na każdym etapie w jakich kierunkach rozwija się projekt i włącza się w proces jego wytwarzania, przekazując swoje uwagi i komentarze (feedback). Następnie te uwagi ponownie są wysłuchane, analizowane i uwzględnione - cykl powtarza się.

### **Manifest programowania zwinnego**

<https://agilemanifesto.org/iso/pl/manifesto.html>

Cztery wartości zwinnego wytwarzania oprogramowania:

- Osoby i interakcje **ponad** procesy i narzędzia
- Działające oprogramowanie **ponad** kompleksową dokumentację
- Współpraca z klientem **ponad** negocjacje kontraktowe
- Reagowanie na zmiany **ponad** sztywne przestrzeganie planu

## Najpopularniejsze frameworki i metodyki zwinne

Podejście zwinne obejmuje co najmniej kilkanaście frameworków i metodyk zwinnych oraz kilkadziesiąt praktyk, wiele technik i różnorodnych narzędzi, m.in.:

- **SCRUM**
- **KANBAN**
- **Lean Software Development**
- **Test-Driven Development (TDD)**

## Podstawowe różnice pomiędzy podejściem zwinnym i tradycyjnym

| ZWINNE                                                    | TRADYCYJNE                                                            |
|-----------------------------------------------------------|-----------------------------------------------------------------------|
| Rezultatem jest oprogramowanie, którego klient oczekuje   | Rezultatem jest oprogramowanie opisane w dokumentacji projektowej     |
| Projekt podzielony na krótkie etapy                       | Projekt podzielony na długie etapy                                    |
| Wyniki biznesowe, działający produkt                      | Procesy, zadania i czynności                                          |
| Interakcje i zarządzanie wiedzą                           | Planowanie i harmonogramowanie                                        |
| Szybkie efekty w porcjach, stopniowe oddawanie przyrostów | Ostateczny efekt na koniec projektu, cały system wdrażany jednorazowo |
| Łatwe wprowadzanie korekt i zmian                         | Wprowadzanie korekt utrudnione                                        |
| Niższe koszty rezygnacji z projektu                       | Wysokie koszty rezygnacji z projektu                                  |
| Mniejsze ryzyko niepowodzenia                             | Większe ryzyko niepowodzenia                                          |
| Osiągnięcie określonych korzyści biznesowych              | Opracowanie systemu o zdefiniowanych cechach                          |

## Bibliografia

---

- Alston J. M., Rich J. A., A Beginner's Guide to Conducting Reproducible Research, 2021, [https://www.researchgate.net/publication/348543052\\_A\\_Beginner's\\_Guide\\_to\\_Conducting\\_Reproducible\\_Research#fullTextFileContent](https://www.researchgate.net/publication/348543052_A_Beginner's_Guide_to_Conducting_Reproducible_Research#fullTextFileContent)
- Farley D., Nowoczesna inżynieria oprogramowania. Stosowanie skutecznych technik szybszego rozwoju oprogramowania wyższej jakości, Helion 2023
- Hoover D. H., Oshineye A., Praktyka czyni mistrza. Wzorce, inspiracje i praktyki rzemieślników programowania, Helion 2017
- Madeyski L., Kitchenham B., Reproducible Research – What, Why and How, 2017, <https://madeyski.e-informatyka.pl/download/MadeyskiKitchenham15.pdf>
- Przewodnik po Scrumie - czym jest Scrum, jak działa i jak zacząć, <https://www.atlassian.com/pl/agile/scrum>
- Rehkopf M., Porównanie Agile i Scrum. Jak wybrać najlepszą metodologię dla siebie, <https://www.atlassian.com/pl/agile/scrum/agile-vs-scrum>
- Silge J., Robinson D., Text Mining with R: A Tidy Approach, O'Reilly 2024
- Śmiałek M., Rybiński K., Inżynieria oprogramowania w praktyce. Od wymagań do kodu z językiem UML, Helion 2024
- Wrycza S., Maślankowski J., Informatyka ekonomiczna. Teoria i zastosowania, PWN 2019
- Zumel N., Mount J., Język R i analiza danych w praktyce, Helion 2021